BANKING DATA ANALYSIS

PROJECT OVERVIEW. This project focuses on analyzing banking transaction data to understand customer behavior, transaction trends, and risk patterns. Various data analysis techniques were applied using Python and Jupyter Notebook to extract meaningful insights from the dataset

PROJECT OBJECTIVE

To clean and preprocess raw transaction data

To analyze credit and debit transaction patterns

To categorize customers based on activity level

To identify risky customer behavior

To perform hypothesis testing on account balance and transaction activity

DATASET OVERVIEW

The dataset contains banking transaction details of customers including transaction amount, account balance, transaction type, customer information, and risk-related attributes

KEY COLUMNS

- TransactionID
- CustomerID
- AccountID
- TransactionType
- TransactionAmount
- AccountBalance
- TransactionDate
- RiskScore
- CreditRating

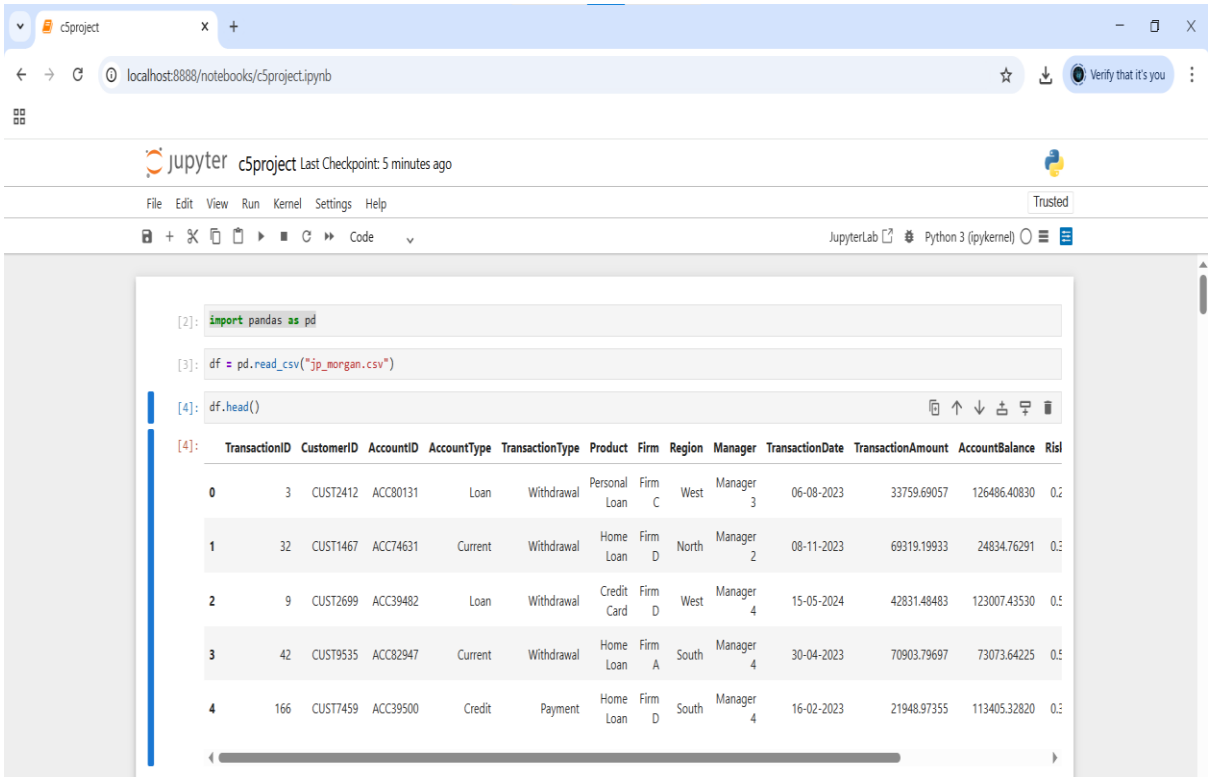
TASK 1: DATA CLEANING

DESCRIPTION:

The dataset was loaded into Jupyter Notebook and cleaned by removing unwanted symbols such as ₹ and commas from the transaction amount column. Data types were corrected by converting transaction amounts into numeric format and transaction dates into datetime format

INSIGHTS:

- The dataset is now clean and consistent
- Data types are correctly formatted
- No missing or invalid values are present
- The dataset is ready for further analysis



	TransactionID	CustomerID	AccountID	AccountType	TransactionType	Product	Firm	Region	Manager	TransactionDate	TransactionAmount	AccountBalance	Risl
0	3	CUST2412	ACC80131	Loan	Withdrawal	Personal Loan	Firm C	West	Manager 3	06-08-2023	33759.69057	126486.40830	0.2
1	32	CUST1467	ACC74631	Current	Withdrawal	Home Loan	Firm D	North	Manager 2	08-11-2023	69319.19933	24834.76291	0.3
2	9	CUST2699	ACC39482	Loan	Withdrawal	Credit Card	Firm D	West	Manager 4	15-05-2024	42831.48483	123007.43530	0.5
3	42	CUST9535	ACC82947	Current	Withdrawal	Home Loan	Firm A	South	Manager 4	30-04-2023	70903.79697	73073.64225	0.5
4	166	CUST7459	ACC39500	Credit	Payment	Home Loan	Firm D	South	Manager 4	16-02-2023	21948.97355	113405.32820	0.3

c5project

localhost:8888/notebooks/c5project.ipynb

Verify that it's you

jupyter c5project Last Checkpoint: 13 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

[6]:
```
df.isnull().sum()
```

[6]:
```
TransactionID       0
CustomerID          0
AccountID           0
AccountType         0
TransactionType     0
Product             0
Firm                0
Region              0
Manager             0
TransactionDate     0
TransactionAmount   0
AccountBalance      0
RiskScore           0
CreditRating        0
TenureMonths        0
dtype: int64
```

[7]:
```
df.columns
```

[7]:
```
Index(['TransactionID', 'CustomerID', 'AccountID', 'AccountType',
       'TransactionType', 'Product', 'Firm', 'Region', 'Manager',
       'TransactionDate', 'TransactionAmount', 'AccountBalance', 'RiskScore',
       'CreditRating', 'TenureMonths'],
      dtype='object')
```

[9]:
```
df['TransactionAmount'] = df['TransactionAmount'].replace('[₹,]', '', regex=True).astype(float)
```

[12]:
```
df['AccountBalance'] = df['AccountBalance'].replace('[₹,]', '', regex=True).astype(float)
```

[14]:
```
df['TransactionDate'] = pd.to_datetime(df['TransactionDate'], dayfirst=True)
```

c5project

localhost:8888/notebooks/c5project.ipynb

Verify that it's you

jupyter c5project Last Checkpoint: 15 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

```
       'CreditRating', 'TenureMonths'],
      dtype='object')
```

[9]:
```
df['TransactionAmount'] = df['TransactionAmount'].replace('[₹,]', '', regex=True).astype(float)
```

[12]:
```
df['AccountBalance'] = df['AccountBalance'].replace('[₹,]', '', regex=True).astype(float)
```

[14]:
```
df['TransactionDate'] = pd.to_datetime(df['TransactionDate'], dayfirst=True)
```

[16]:
```
df['TransactionType'] = df['TransactionType'].str.lower().str.strip()
df['AccountType'] = df['AccountType'].str.lower().str.strip()
df['Region'] = df['Region'].str.lower().str.strip()
```

[17]:
```
df = df.drop_duplicates()
```

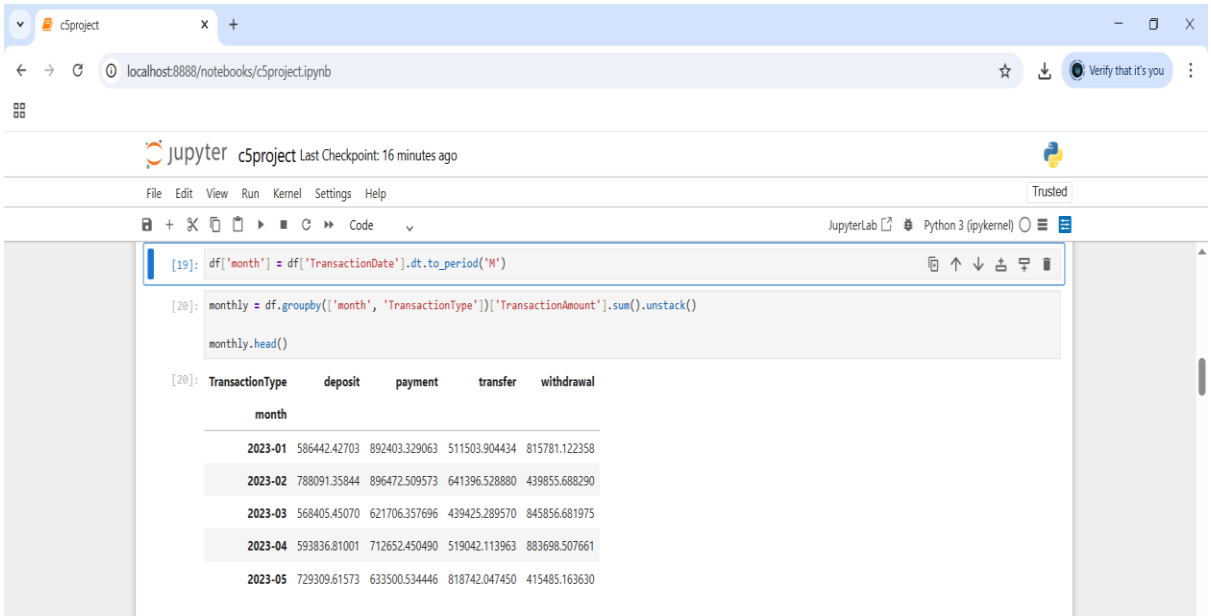
TASK 2: TRANSACTION ANALYSIS

DESCRIPTION:

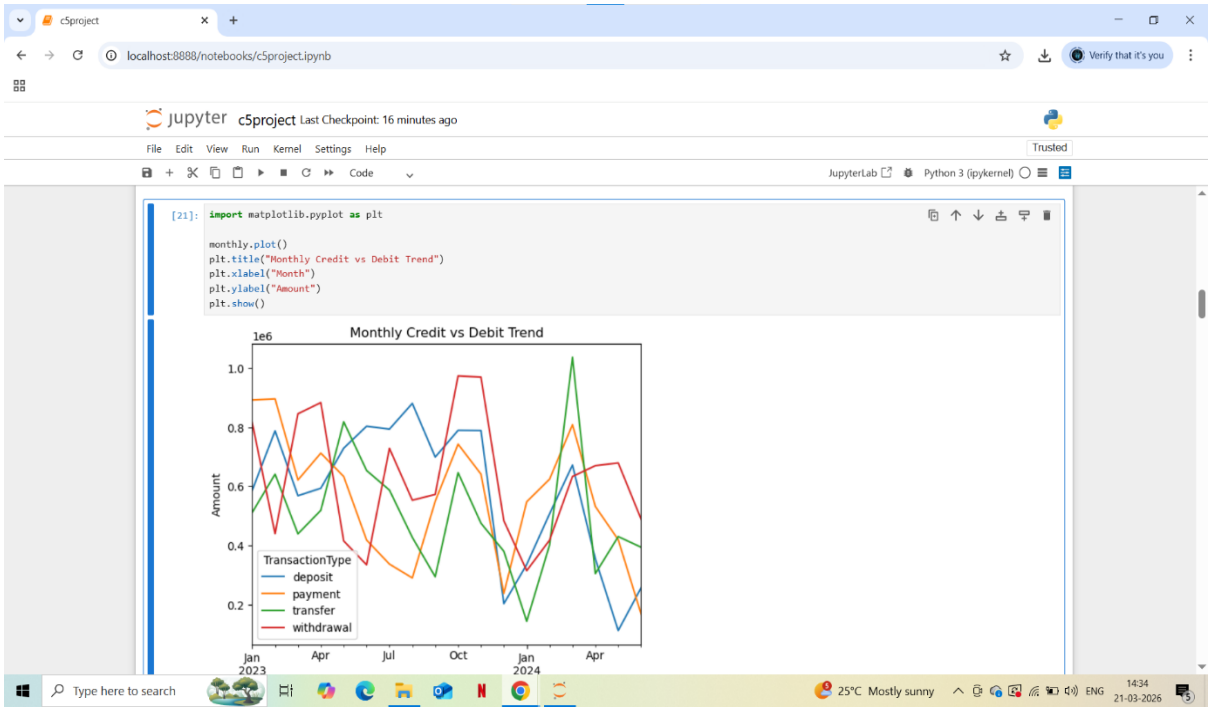
Monthly credit and debit transactions were analyzed by grouping the dataset based on month and transaction type. A visualization was created to compare trends over time.

INSIGHTS:

- Debit transactions are higher than credit in some months
- This indicates increased customer spending behavior
- Transaction patterns vary across different months
- Monthly trends help understand financial activity



TransactionType	deposit	payment	transfer	withdrawal
month				
2023-01	586442.42703	892403.329063	511503.904434	815781.122358
2023-02	788091.35844	896472.509573	641396.528880	439855.688290
2023-03	568405.45070	621706.357696	439425.289570	845856.681975
2023-04	593836.81001	712652.450490	519042.113963	883698.507661
2023-05	729309.61573	633500.534446	818742.047450	415485.163630


c5project
localhost:8888/notebooks/c5project.ipynb
jupyter c5project Last Checkpoint: 16 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[21]:
import matplotlib.pyplot as plt
monthly.plot()
plt.title("Monthly Credit vs Debit Trend")
plt.xlabel("Month")
plt.ylabel("Amount")
plt.show()
Monthly Credit vs Debit Trend
1e6
1.0
0.8
0.6
0.4
0.2
Amount
TransactionType
deposit
payment
transfer
withdrawal
Jan 2023
Apr
Jul
Oct
Jan 2024
Apr
Type here to search
25°C Mostly sunny
ENG
14:34
21-03-2026

c5project

localhost:8888/notebooks/c5project.ipynb

jupyter c5project Last Checkpoint: 17 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

[22]:
```
account_summary = df.groupby('AccountID')['TransactionAmount'].sum()

top_accounts = account_summary.sort_values(ascending=False).head(10)
bottom_accounts = account_summary.sort_values().head(10)

top_accounts
```

[22]:
```
AccountID
ACC42710    539494.876440
ACC45907    532556.635574
ACC81631    483854.558670
ACC32627    471468.023823
ACC18140    458219.210460
ACC95774    453488.129360
ACC55331    448188.410346
ACC29477    443067.484857
ACC29646    439477.177961
ACC22036    437626.170698
Name: TransactionAmount, dtype: float64
```

[23]:
```
bottom_accounts
```

[23]:
```
AccountID
ACC12334    15159.688345
ACC50439    24088.157284
ACC65144    30425.901148
ACC28305    32162.348245
ACC43771    32819.585640
ACC49180    37023.532950
ACC46655    37365.268690
ACC30146    42545.848430
ACC39482    42831.484830
ACC87006    42903.165440
Name: TransactionAmount, dtype: float64
```

Type here to search 25°C Mostly sunny ENG 14:35 21-03-2026

c5project

localhost:8888/notebooks/c5project.ipynb

jupyter c5project Last Checkpoint: 18 minutes ago

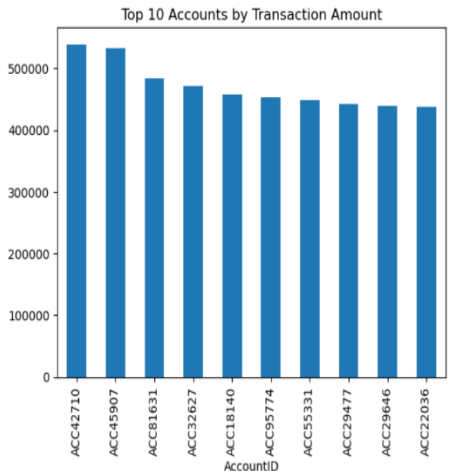
File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

[24]:
```
top_accounts.plot(kind='bar')
plt.title("Top 10 Accounts by Transaction Amount")
plt.show()
```



Type here to search 25°C Mostly sunny ENG 14:36 21-03-2026

c5project

localhost:8888/notebooks/c5project.ipynb

jupyter c5project Last Checkpoint: 19 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

[25]:
```
df = df.sort_values(['AccountID', 'TransactionDate'])

df['gap'] = df.groupby('AccountID')['TransactionDate'].diff().dt.days
```

[26]:
```
dormant_accounts = df[df['gap'] > 60]

dormant_accounts.head()
```

[26]:

tType	TransactionType	Product	Firm	Region	Manager	TransactionDate	TransactionAmount	AccountBalance	RiskScore	CreditRating	TenureMonths	month	gap
credit	deposit	Savings Account	Firm B	south	Manager 1	2023-07-01	81608.80093	67810.12401	0.505365	694	103	2023-07	161.0
credit	deposit	Personal Loan	Firm C	north	Manager 4	2023-10-19	92323.65567	39827.55468	0.672591	768	198	2023-10	110.0
credit	transfer	Home Loan	Firm E	north	Manager 4	2023-08-22	103375.34420	51672.12356	0.813273	341	32	2023-08	103.0
loan	withdrawal	Mutual Fund	Firm E	west	Manager 1	2023-12-25	35748.04389	105604.12180	0.142407	304	16	2023-12	125.0
vings	payment	Personal Loan	Firm B	north	Manager 1	2023-08-17	61343.12824	110972.96240	0.306461	329	177	2023-08	163.0

c5project

localhost:8888/notebooks/c5project.ipynb

jupyter c5project Last Checkpoint: 19 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

credit	deposit	Loan	C	north	4	2023-10-19	92323.65567	39827.55468	0.672591	768	198	10	110.0
credit	transfer	Home Loan	Firm E	north	Manager 4	2023-08-22	103375.34420	51672.12356	0.813273	341	32	2023-08	103.0
loan	withdrawal	Mutual Fund	Firm E	west	Manager 1	2023-12-25	35748.04389	105604.12180	0.142407	304	16	2023-12	125.0
vings	payment	Personal Loan	Firm B	north	Manager 1	2023-08-17	61343.12824	110972.96240	0.306461	329	177	2023-08	163.0

[27]:
```
df['year'] = df['TransactionDate'].dt.year

yearly = df.groupby(['year', 'TransactionType'])['TransactionAmount'].sum().unstack()

yearly
```

[27]:

TransactionType	deposit	payment	transfer	withdrawal
year				
2023	8.225787e+06	6.974170e+06	6.395137e+06	8.017619e+06
2024	2.238946e+06	3.100943e+06	2.711474e+06	3.205223e+06

TASK 3: CUSTOMER PROFILING

DESCRIPTION:

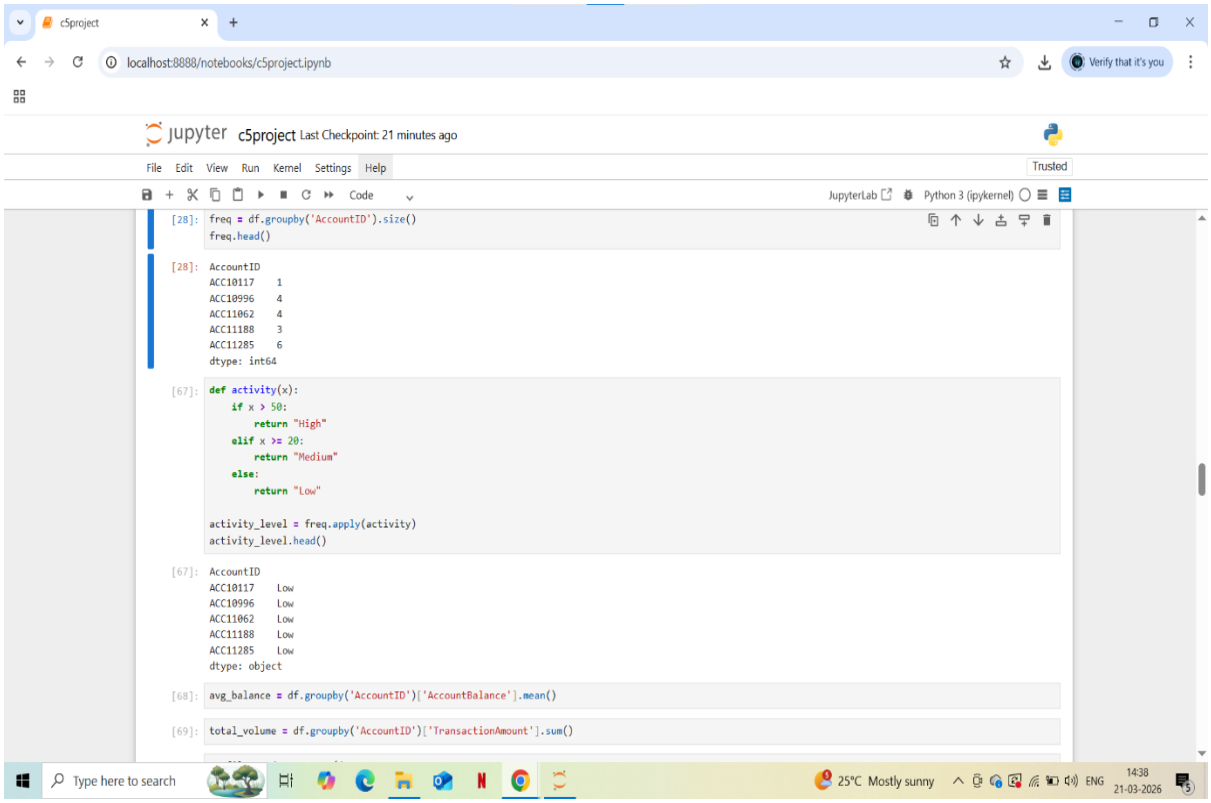
Customers were categorized based on transaction frequency into High, Medium, and Low activity groups.

RUBRIC USED:

- High Activity: ≥ 5 transactions
- Medium Activity: 3–4 transactions
- Low Activity: ≤ 2 transactions

INSIGHTS:

- Customers are successfully categorized into activity levels
- High activity customers generate more transaction volume
- Some customers show high frequency but low balance
- These accounts may indicate potential financial risk



c5project

localhost:8888/notebooks/c5project.ipynb

jupyter c5project Last Checkpoint: 23 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

[70]:
```
profile = pd.DataFrame({
    'Frequency': freq,
    'ActivityLevel': activity_level,
    'AvgBalance': avg_balance,
    'TotalVolume': total_volume
})

profile.head()
```

[70]:

AccountID	Frequency	ActivityLevel	AvgBalance	TotalVolume
ACC10117	1	Low	90780.256640	56317.920060
ACC10996	4	Low	64046.568590	223757.534516
ACC11062	4	Low	62784.100737	265928.583340
ACC11188	3	Low	80558.926400	116557.954500
ACC11285	6	Low	95745.546255	373189.528410

[71]:
```
high_inflow = profile.sort_values(by='TotalVolume', ascending=False).head(10)
```

[72]:
```
hf_lb = profile[(profile['Frequency'] > 50) & (profile['AvgBalance'] < 5000)]
```

[73]:
```
low_balance = profile[profile['AvgBalance'] <= 1000]
```

c5project

localhost:8888/notebooks/c5project.ipynb

jupyter c5project Last Checkpoint: 24 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

[74]:
```
profile['ActivityLevel'].value_counts().plot(kind='bar')
plt.title("Customer Activity Levels")
plt.show()
```

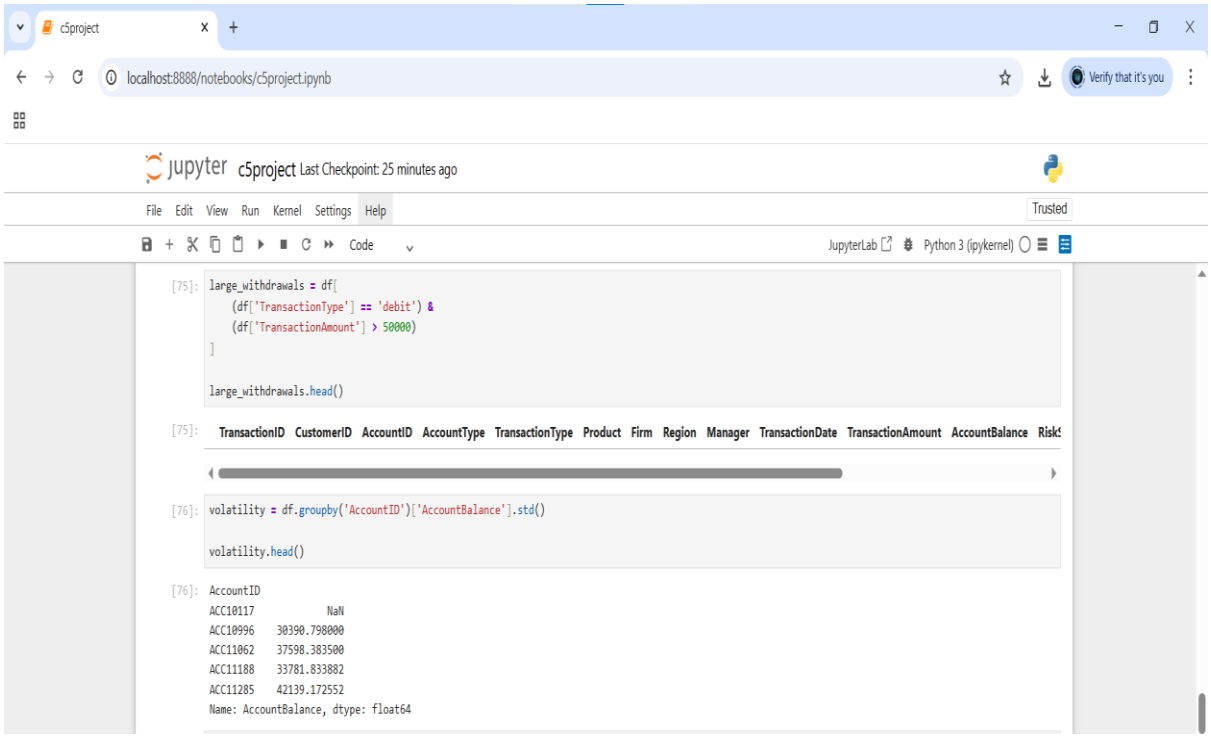
TASK 4: RISK ANALYSIS

DESCRIPTION:

Risk analysis was performed to identify customers with unusual transaction patterns, particularly those with high transaction frequency and low account balance.

INSIGHTS:

- High frequency with low balance accounts are risky
- Such accounts may indicate financial instability
- These customers require monitoring
- Risk analysis helps in better decision-making



c5project

localhost:8888/notebooks/c5project.ipynb

Verify that it's you

jupyter c5project Last Checkpoint: 26 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

[77]:
```
high_volatility = volatility.sort_values(ascending=False).head(10)
high_volatility
```

[77]:
```
AccountID
ACC39529    87925.613732
ACC78589    74986.233658
ACC49774    67777.809369
ACC28612    59940.470436
ACC57516    57794.003514
ACC51971    56709.662822
ACC33287    55392.907529
ACC74656    55037.331771
ACC66190    52142.885192
ACC90887    51988.111482
Name: AccountBalance, dtype: float64
```

c5project

localhost:8888/notebooks/c5project.ipynb

jupyter c5project Last Checkpoint: 27 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

[78]:
```
Q1 = df['TransactionAmount'].quantile(0.25)
Q3 = df['TransactionAmount'].quantile(0.75)

IQR = Q3 - Q1

outliers = df[
    (df['TransactionAmount'] < Q1 - 1.5 * IQR) |
    (df['TransactionAmount'] > Q3 + 1.5 * IQR)
]

outliers.head()
```

[78]:

	TransactionID	CustomerID	AccountID	AccountType	TransactionType	Product	Firm	Region	Manager	TransactionDate	TransactionAmount	AccountBalance	R
97	67	CUST6391	ACC26940	current	payment	Home Loan	Firm A	south	Manager 4	2023-03-16	-25964.94612	94822.29734	
129	161	CUST4373	ACC41829	current	payment	Savings Account	Firm A	north	Manager 3	2024-03-22	135972.34390	42302.00951	-
701	191	CUST6837	ACC46655	current	deposit	Mutual Fund	Firm E	west	Manager 2	2024-06-16	-29563.97803	49810.32678	
53	178	CUST1189	ACC50439	savings	deposit	Savings Account	Firm D	east	Manager 1	2024-06-16	-60979.07348	57916.06101	
659	21	CUST1644	ACC77533	credit	payment	Personal Loan	Firm D	south	Manager 4	2023-12-24	-30826.73980	68006.03943	

[79]:
```
risky_accounts = set(large_withdrawals['AccountID']) | set(outliers['AccountID'])
len(risky_accounts)
```

[79]: 9

Type here to search

25°C Mostly sunny ENG 14:44 21-03-2026

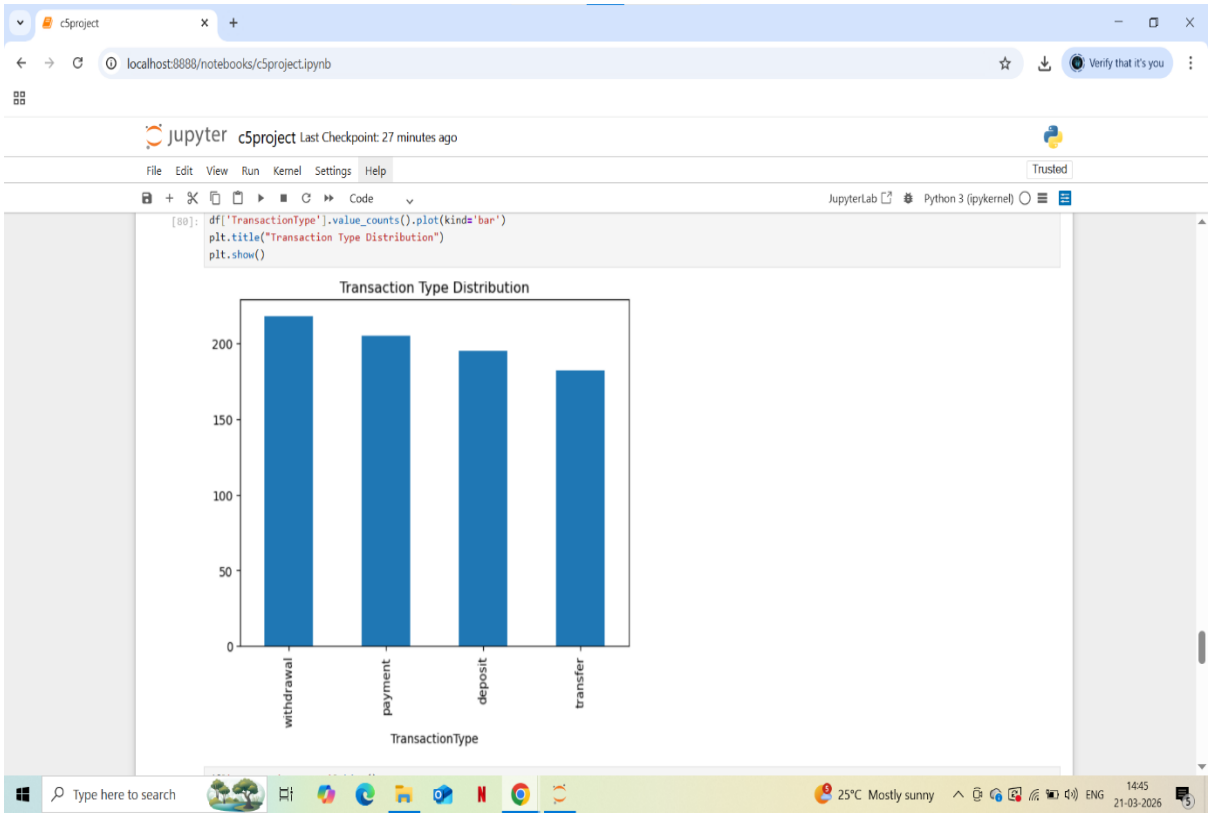
TASK 5: MANAGER / REGION PERFORMANCE ANALYSIS

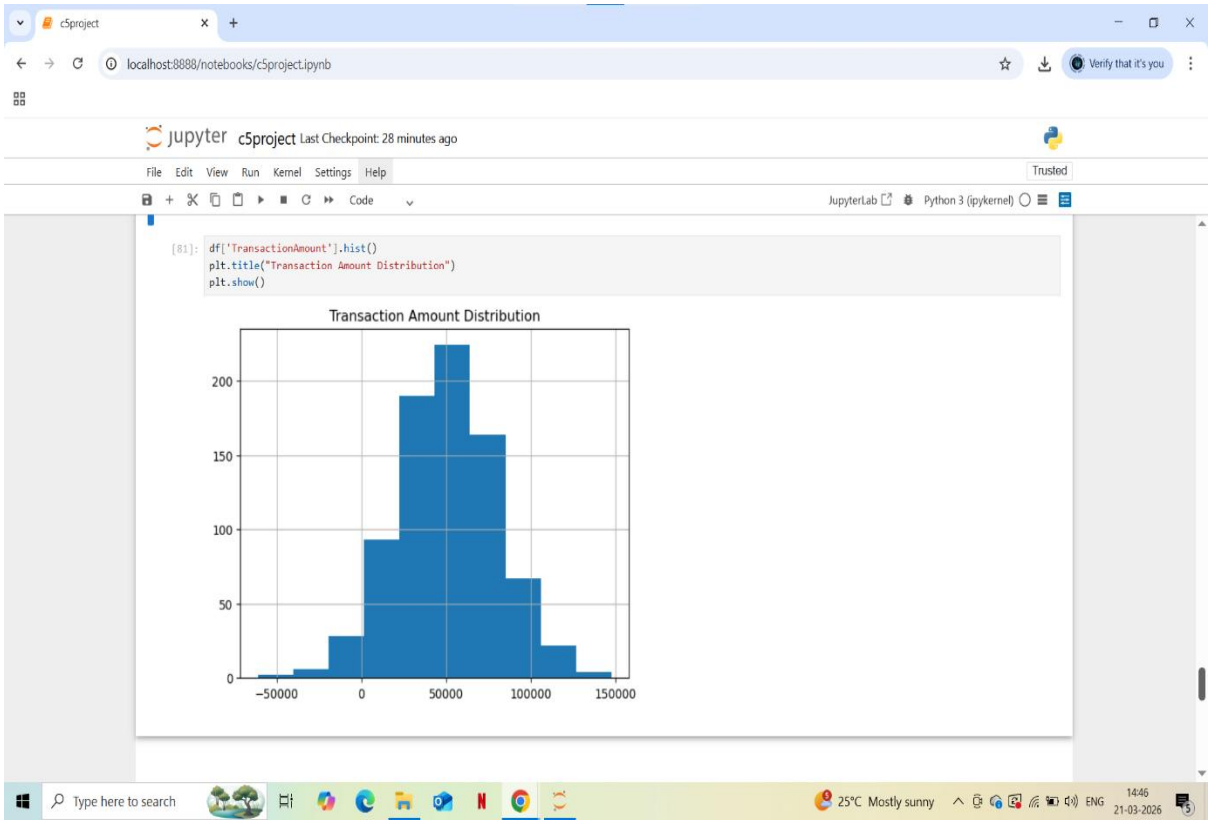
DESCRIPTION:

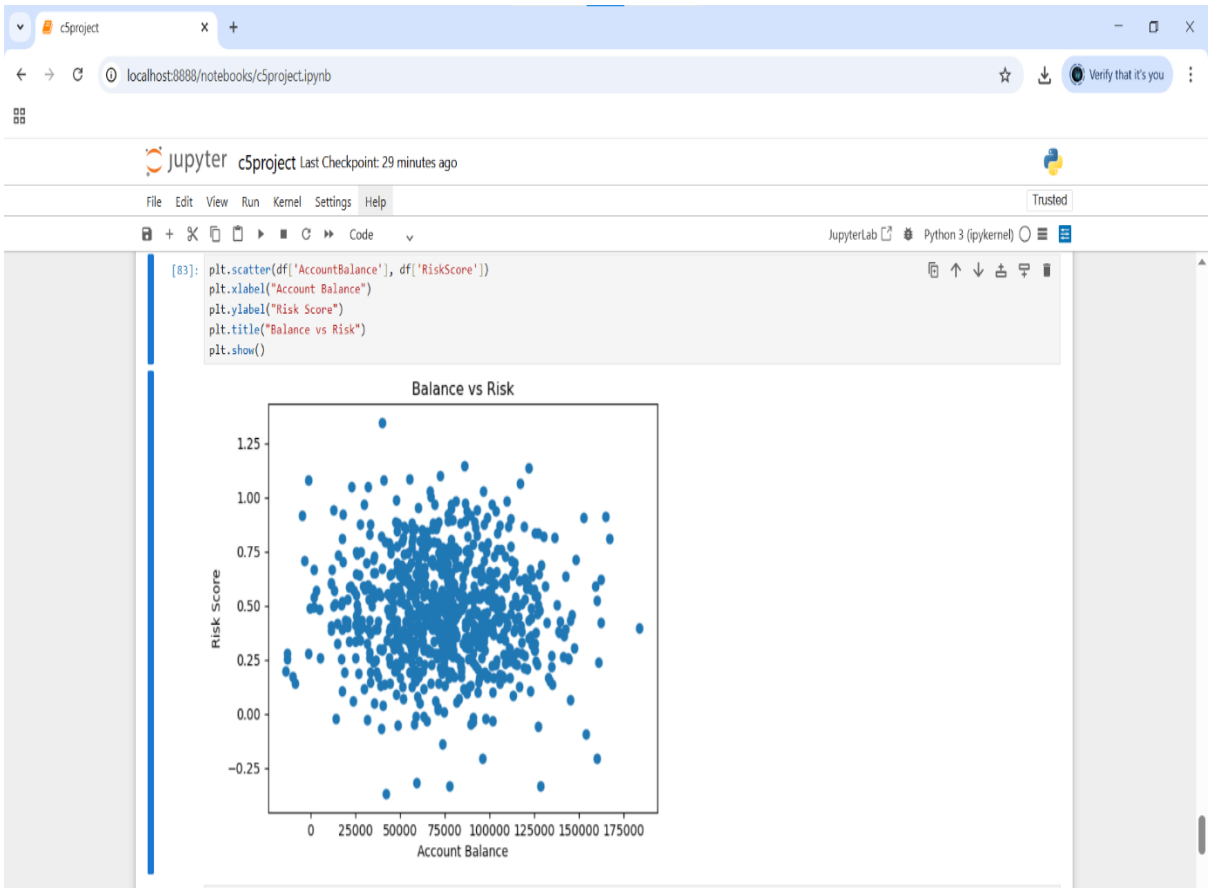
In this task, the performance of managers and regions was analyzed by evaluating total transaction volume and average account balance. The data was grouped based on Manager and Region to identify top-performing and underperforming segments.

INSIGHTS:

- Some managers handle higher transaction volumes
- Certain regions show stronger financial activity
- Performance varies across managers and regions
- High-performing regions contribute more to business growth




c5project
localhost:8888/notebooks/c5project.ipynb
jupyter c5project Last Checkpoint: 28 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[81]: df['TransactionAmount'].hist()
plt.title("Transaction Amount Distribution")
plt.show()
Transaction Amount Distribution
200
150
100
50
0
−50000
0
50000
100000
150000
Type here to search
25°C Mostly sunny
ENG
14:46
21-03-2026


c5project
localhost:8888/notebooks/c5project.ipynb
jupyter c5project Last Checkpoint: 29 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[83]: plt.scatter(df['AccountBalance'], df['RiskScore'])
plt.xlabel("Account Balance")
plt.ylabel("Risk Score")
plt.title("Balance vs Risk")
plt.show()
Balance vs Risk
Risk Score
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
0
25000
50000
75000
100000
125000
150000
175000
Account Balance

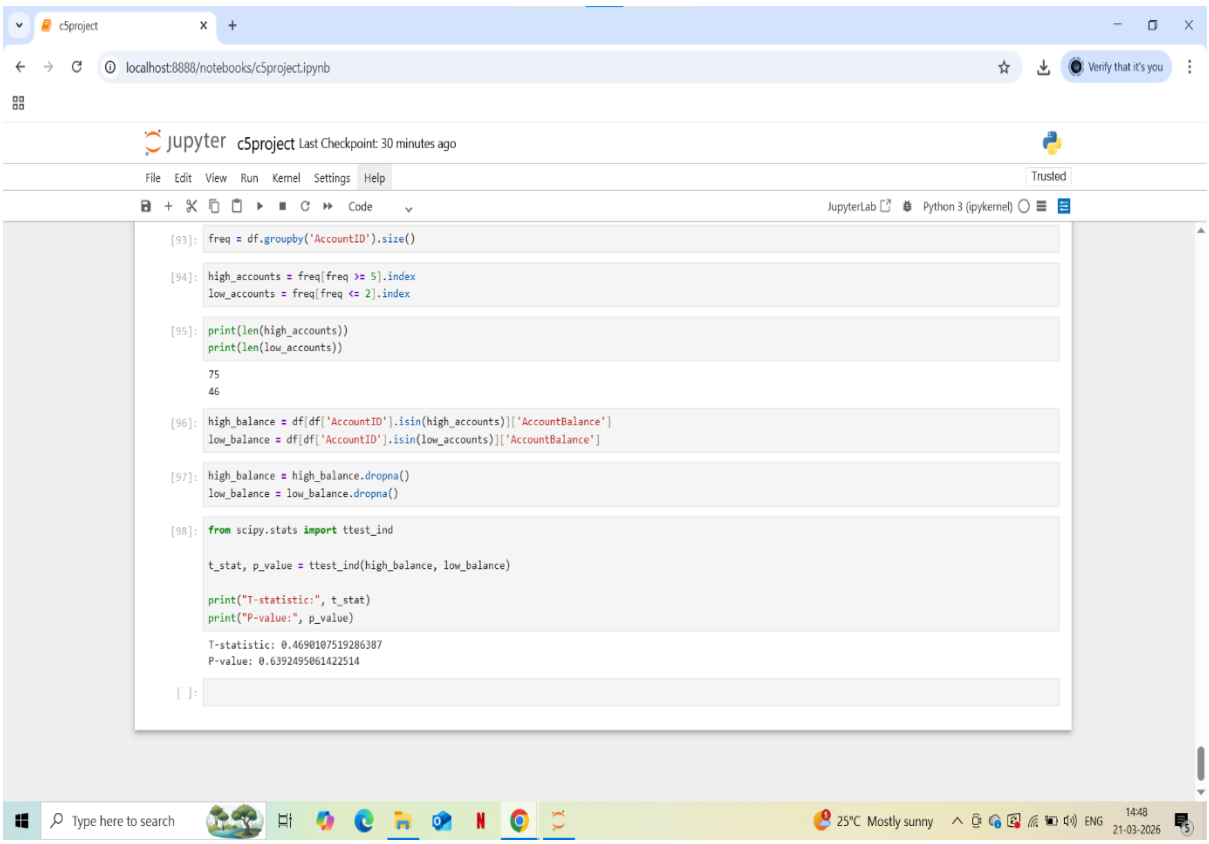
TASK 6: HYPOTHESIS TESTING

DESCRIPTION:

A t-test was conducted to compare the average account balance between high activity and low activity customers.

RESULT:

- T-statistic: 0.469
- P-value: 0.639

INSIGHTS:

- The p-value (0.639) is greater than 0.05
- There is no significant difference between high and low activity accounts
- Transaction frequency does not strongly impact account balance
- The null hypothesis is accepted